



Chapter 4: Conditional Structure

o4o2231o1 Computer Programming for Mathematics 1 – Department of Mathematics, Faculty of Applied Science, King Mongkut's University of Technology North Bangkok

Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul | Semester 1, Academic Year 2026

Learning Objectives

Upon completing this chapter, students will be able to:

O1

Conditional Expressions

Use comparison operators and logical operators to construct conditional expressions that evaluate to `True` or `False`.

O2

if / if-else / if-elif-else

Write `if`, `if-else`, and `if-elif-else` statements to control program flow based on conditions.

O3

Indentation

Explain the importance of indentation in Python for defining the scope of a code block.

O4

Nested if

Write nested `if` statements to solve multi-level decision problems.

O5

Mathematical Applications

Apply conditional structures to solve mathematical problems such as triangle classification and leap year detection.

 CLO: This chapter's content aligns with Course Learning Outcome 3 (CLO 3).



Why Do We Need Decision Structures?

Up to this point, our programs have executed sequentially – one line at a time, from top to bottom. However, real-world problems often require the program to **make decisions** and choose different actions depending on conditions.

Sequential Execution

Every statement runs in order, regardless of any condition.
Simple but limited in capability.

Conditional Execution

The program evaluates a condition and chooses which path to follow – enabling intelligent, dynamic behavior.

Real-World Example

If a student's score meets the threshold, they pass. Otherwise, they fail.
This simple logic requires a conditional structure.

Conditional Expressions

A **conditional expression** (condition) is an expression that evaluates to a Boolean value: `True` or `False`. These are the building blocks of all decision structures in Python. Conditions are constructed using **comparison operators**.

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	Not equal to	<code>5 != 3</code>	<code>True</code>
<code>></code>	Greater than	<code>5 > 8</code>	<code>False</code>
<code><</code>	Less than	<code>5 < 8</code>	<code>True</code>
<code>>=</code>	Greater than or equal to	<code>5 >= 5</code>	<code>True</code>
<code><=</code>	Less than or equal to	<code>3 <= 2</code>	<code>False</code>

Common Pitfall: = vs ==

Assignment Operator =

Used to **assign a value** to a variable. The variable on the left receives the value on the right.

```
x = 10 # x now holds the value 10
```

This is a **statement**, not a comparison. It does not produce True or False.

Comparison Operator ==

Used to **compare two values** and check if they are equal. Returns a Boolean result.

```
x == 10 # True if x equals 10, False otherwise
```

This is an **expression** that evaluates to True or False and is used inside conditions.

⚠ Warning: Confusing = (assignment) with == (comparison) is one of the most common beginner mistakes. They have completely different meanings and cannot be used interchangeably.

SECTION 4.2

Logical Operators

When multiple conditions must be combined, Python provides three **logical operators**: `and`, `or`, and `not`. These allow you to build complex conditional expressions from simpler ones.

`and`

Returns `True` only if **both** conditions are true. If either condition is false, the result is false.

Example: `age >= 18 and income >= 10000`

`or`

Returns `True` if **at least one** condition is true. Returns false only when both are false.

Example: `age < 15 or income > 30000`

`not`

Inverts the Boolean value of a condition. Turns `True` into `False` and vice versa.

Example: `not (age >= 18)`

Logical Operators – Code Example

File: 04logical.py

Source Code

```
age = 20
income = 15000

print(age >= 18 and income >= 10000)
print(age < 15 or income > 30000)
print(not (age >= 18))
```

Output

```
True
False
False
```

Explanation

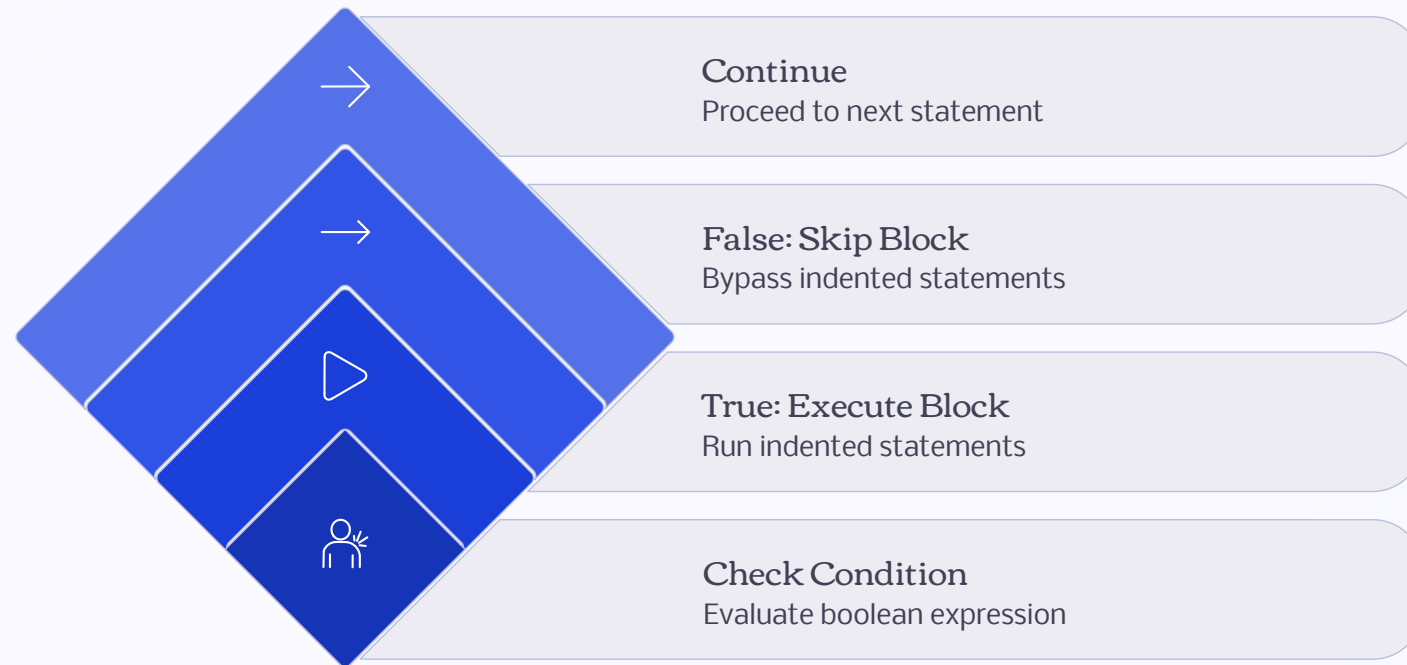
- `age >= 18 and income >= 10000`: Both `20 ≥ 18` and `15000 ≥ 10000` are true → `True`
- `age < 15 or income > 30000`: `20 < 15` is false and `15000 > 30000` is false → `False`
- `not (age >= 18)`: `20 ≥ 18` is true, negated → `False`

The if Statement

The if statement executes the block of code inside it **only when the condition is true**. The syntax is:

```
if condition:  
    statement_1  
    statement_2  
    ...
```

The block of statements that belongs to the if is defined by **indentation** – typically 4 spaces. Statements that are not indented are outside the block and always execute.



This diagram illustrates how Python evaluates the condition and either enters the indented block or skips it entirely before continuing with the rest of the program.

The if Statement – Code Example

File: 04if.py

Source Code

```
score = 75

if score >= 50:
    print("ผ่านเกณฑ์")
    print("ยินดีด้วย")

print("จบโปรแกรม")
```

Output

```
ผ่านเกณฑ์
ยินดีด้วย
จบโปรแกรม
```

Key Observation

The line `print("จบโปรแกรม")` is **not indented**, so it is outside the if block and executes regardless of the condition. Only the two indented `print` statements are conditional.

- ❗ In Python, indentation is **mandatory** and defines block scope – unlike other languages that use curly braces `{}`. Inconsistent indentation will cause a syntax error.

The Importance of Indentation in Python

Indentation Defines Block Scope

In Python, there are no curly braces `{}` to mark the beginning and end of a block. Instead, all statements at the same indentation level belong to the same block.

Standard Convention: 4 Spaces

The Python community standard (PEP 8) recommends using **4 spaces** per indentation level. Mixing tabs and spaces will cause errors.

Indentation Errors Are Fatal

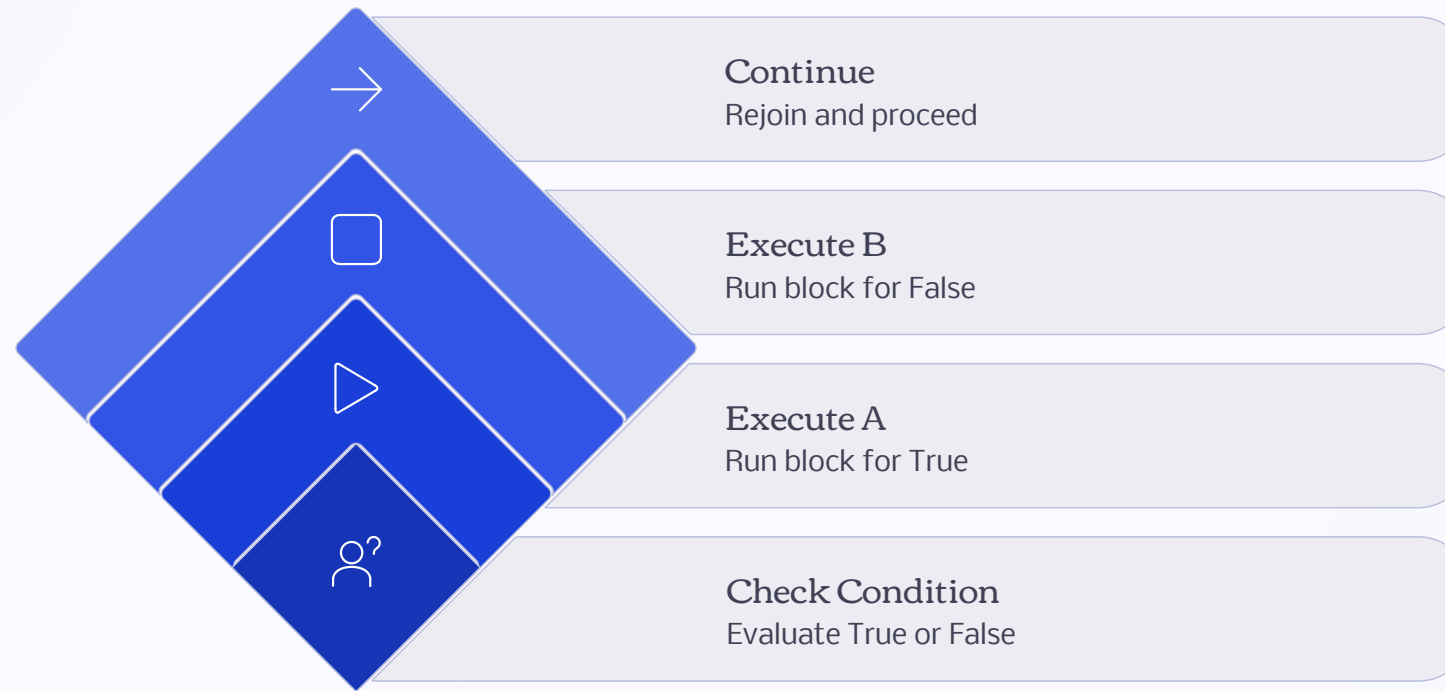
An `IndentationError` will prevent the program from running at all. Python is strict – every block must be consistently indented.



The if-else Statement

When a program must choose between **exactly two paths**, use the `else` clause to specify what happens when the condition is false.

```
if condition:  
    # executed when condition is True  
    statement_A  
else:  
    # executed when condition is False  
    statement_B
```



Exactly one of the two blocks will always execute – never both, never neither. This guarantees complete coverage of all cases.

if-else – Even or Odd Number

File: 04ifelse.py

Source Code

```
num = int(input("ใส่จำนวนเต็ม: "))

if num % 2 == 0:
    print(f"{num} เป็นจำนวนคู่")
else:
    print(f"{num} เป็นจำนวนคี่")
```

How It Works

The modulo operator `%` returns the remainder of division. If `num % 2 == 0`, the number divides evenly by 2 and is even. Otherwise, it is odd.

Sample Output

ใส่จำนวนเต็ม: 7
7 เป็นจำนวนคี่

Another Run

ใส่จำนวนเต็ม: 4
4 เป็นจำนวนคู่

The if-elif-else Statement

When there are **more than two alternatives**, use `elif` (short for "else if") to check additional conditions in sequence. Python evaluates from top to bottom and executes **only the first block whose condition is true**. All remaining branches are skipped.

```
if condition_1:  
    block_1  
elif condition_2:  
    block_2  
elif condition_3:  
    block_3  
else:  
    block_default
```

- ❶ The `else` clause at the end is optional but recommended as a catch-all for any case not covered by the preceding conditions. Only one block will ever execute per run.

if-elif-else – Grade Classification

File: 04grade.py

Source Code

```
score = int(input("ใส่คะแนน: "))

if score >= 80:
    grade = "A"
elif score >= 70:
    grade = "B"
elif score >= 60:
    grade = "C"
elif score >= 50:
    grade = "D"
else:
    grade = "F"

print(f"เกรดที่ได้คือ {grade}")
```

Sample Output

ใส่คะแนน: 73
เกรดที่ได้คือ B

Grading Scale

Score Range	Grade
80 and above	A
70 - 79	B
60 - 69	C
50 - 59	D
Below 50	F

How Python Evaluates elif Chains

1

Check if

Evaluate the first condition. If `True`, execute its block and exit the entire chain.

2

Check elif

Only reached if all previous conditions were `False`. Evaluate this condition next.

3

More elif

Continue checking each `elif` in order. The first one that is `True` wins.

4

Fall to else

If no condition was true, execute the `else` block as the default case.

For a score of 73: `73 >= 80` is `False` → `73 >= 70` is `True` → `grade = "B"` → remaining branches are skipped entirely.

SECTION 4.6

Nested if Statements

A if statement can be placed **inside the block of another**

Nested if – Positive, Negative, or Zero

File: 04nested.py – This example checks whether a number is positive, negative, or zero. If positive, it further checks whether it is even or odd.

Source Code

```
n = int(input("ใส่จำนวนเต็ม: "))

if n > 0:
    print("จำนวนบวก")
    if n % 2 == 0:
        print("และเป็นจำนวนคู่")
    else:
        print("และเป็นจำนวนคี่")
elif n < 0:
    print("จำนวนลบ")
else:
    print("ศูนย์")
```

Sample Output

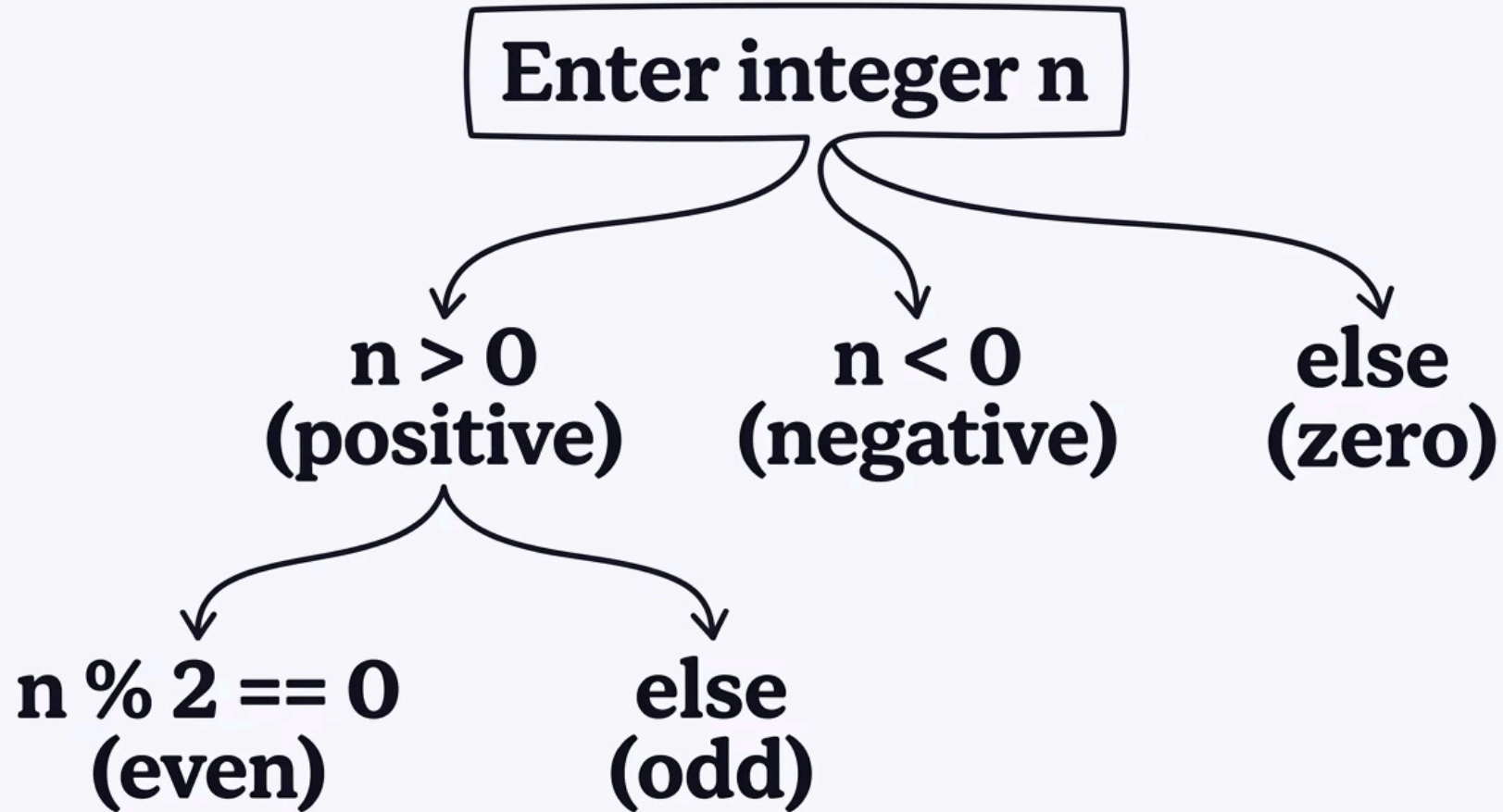
```
ใส่จำนวนเต็ม: 8
จำนวนบวก
และเป็นจำนวนคู่
```

Logic Flow

- If `n > 0`: print "positive", then check even/odd with inner if-else
- If `n < 0`: print "negative" (no inner check needed)
- Otherwise (`n == 0`): print "zero"

Visualizing Nested if Logic

The tree structure clearly shows how the outer condition ($n > 0$) must be true before the inner condition ($n \% 2 == 0$) is ever evaluated. The negative and zero branches have no further sub-decisions.



Applied Example: Triangle Classification

This example demonstrates a practical application of nested if statements: given three side lengths, determine whether they form a valid triangle, and if so, classify it by side type.

Step 1: Validity Check

A valid triangle requires that the sum of any two sides must be greater than the third side: $a+b > c$, $b+c > a$, and $a+c > b$.

Step 2: Equilateral

If all three sides are equal ($a == b == c$), it is an **equilateral triangle** (สามเหลี่ยมด้านเท่า).

Step 3: Isosceles

If exactly two sides are equal ($a==b$ or $b==c$ or $a==c$), it is an **isosceles triangle** (สามเหลี่ยมหน้าจั่ว).

Step 4: Scalene

If all three sides are different, it is a **scalene triangle** (สามเหลี่ยมด้านไม่เท่า).

Triangle Classification – Code

File: 04triangle.py

Source Code

```
a = float(input("ด้าน a: "))
b = float(input("ด้าน b: "))
c = float(input("ด้าน c: "))

if a + b > c and b + c > a and a + c > b:
    if a == b == c:
        print("สามเหลี่ยมด้านเท่า")
    elif a == b or b == c or a == c:
        print("สามเหลี่ยมหน้าจั่ว")
    else:
        print("สามเหลี่ยมด้านไม่เท่า")
else:
    print("ไม่สามารถประกอบเป็นสามเหลี่ยมได้")
```

Sample Output

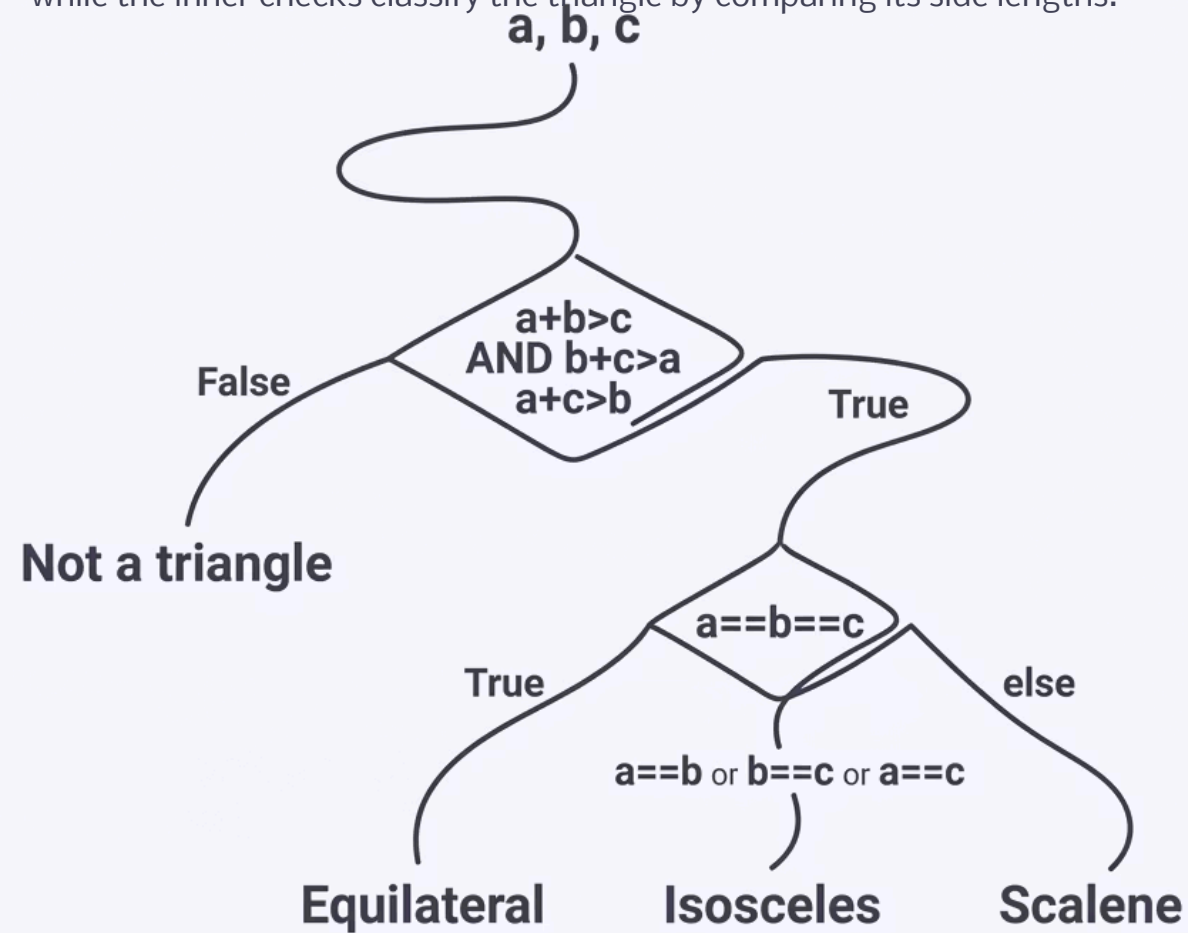
```
ด้าน a: 5
ด้าน b: 5
ด้าน c: 8
สามเหลี่ยมหน้าจั่ว
```

Why This Works

The outer if uses `and` to combine three comparison conditions simultaneously. Only if all three are true does the program proceed to classify the triangle type using nested `if-elif-else`.

Triangle Classification – Logic Flow

This flow diagram illustrates the two-level nested decision structure: the outer check validates the triangle inequality theorem, while the inner checks classify the triangle by comparing its side lengths.



Chapter Summary

1

Conditional Expressions

Built from comparison operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) and logical operators (`and`, `or`, `not`). Always evaluate to `True` or `False`.

2

if / else / elif

if executes when condition is true. else provides an alternative. elif adds multiple levels of choice, evaluated top-to-bottom.

3

Indentation

Mandatory in Python. Defines the scope of each code block. Inconsistent indentation causes errors. Use 4 spaces per level consistently.

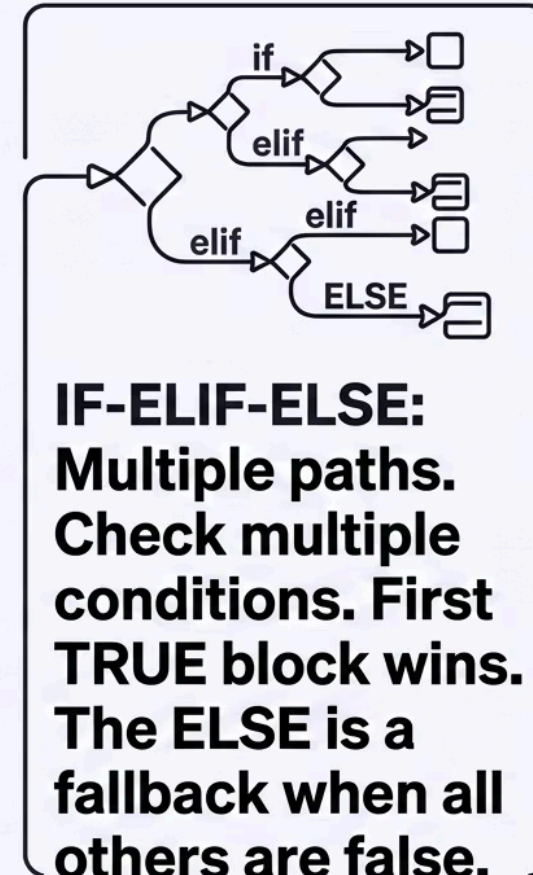
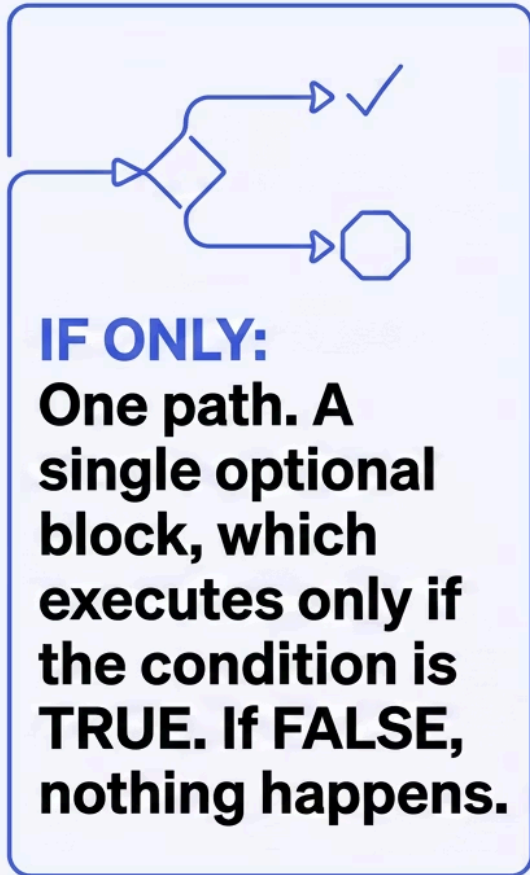
4

Nested if

Placing an if inside another if block enables multi-level decision making for complex, hierarchical problems.



Comparison: if vs if-else vs if-elif-else



Choose the right structure based on how many distinct outcomes your problem has. Use `if` alone for optional actions, `if-else` for binary choices, and `if-elif-else` for multi-way branching.

EXERCISES

End-of-Chapter Exercises

Apply what you have learned in this chapter by writing the following Python programs:

1 Positive, Negative, or Zero

Write a program that accepts one integer and displays whether it is a positive number, a negative number, or zero.

2 Leap Year Checker

Write a program that accepts a year (A.D.) and checks whether it is a leap year. A year is a leap year if it is divisible by 4, *except* for years divisible by 100, which must also be divisible by 400.

3 Maximum of Three Numbers

Write a program that accepts three numbers and displays the largest value among them.

4 BMI Classification

Write a program that accepts a BMI value and displays the category: below 18.5 = Underweight, 18.5 - 24.9 = Normal, 25 - 29.9 = Overweight, 30 and above = Obese.

Exercise 1: Positive, Negative, or Zero

This exercise directly applies the nested `if` concept from Section 4.6. The program reads one integer and classifies it into one of three categories.

Suggested Approach

```
n = int(input("ใส่จำนวนเต็ม: "))

if n > 0:
    print("จำนวนบวก")
elif n < 0:
    print("จำนวนลบ")
else:
    print("ศูนย์")
```

Key Concepts Used

- Comparison operators `>` and `<`
- `if-elif-else` for three mutually exclusive outcomes
- The `else` clause catches the remaining case (zero) without needing an explicit condition

Sample Runs

ใส่จำนวนเต็ม: 5 → จำนวนบวก

ใส่จำนวนเต็ม: -3 → จำนวนลบ

ใส่จำนวนเต็ม: 0 → ศูนย์

Exercise 2: Leap Year Checker

File solution for Exercise 2. The leap year rule requires combining multiple conditions with `and`, `or`, and `not` – a direct application of logical operators from Section 4.2.

Leap Year Rule

- Divisible by 4 → candidate leap year
- But if also divisible by 100 → NOT a leap year
- Unless also divisible by 400 → IS a leap year

Source Code

```
y = int(input("ใส่ปี ค.ศ.: "))

if (y % 4 == 0 and y % 100 != 0) or (y % 400 == 0):
    print(f"{y} เป็นปีอธิกสุรทิน")
else:
    print(f"{y} ไม่เป็นปีอธิกสุรทิน")
```

Sample Output

```
ใส่ปี ค.ศ.: 2024
2024 เป็นปีอธิกสุรทิน
```

Test Cases

Year	÷4?	÷100?	÷400?	Leap?
2024	Yes	No	No	Yes
1900	Yes	Yes	No	No
2000	Yes	Yes	Yes	Yes
2023	No	No	No	No

Exercise 3: Maximum of Three Numbers

File solution for Exercise 3. This exercise demonstrates how to find the maximum value using sequential `if` statements – a classic algorithmic pattern.

Source Code

```
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))

big = a
if b > big:
    big = b
if c > big:
    big = c

print(f"ค่ามากที่สุดคือ {big}")
```

Sample Output

```
a: 4
b: 9
c: 6
ค่ามากที่สุดคือ 9.0
```

Algorithm Explanation

- Start by assuming `a` is the largest: `big = a`
- Compare `b` against `big`; update if larger
- Compare `c` against `big`; update if larger
- After both checks, `big` holds the maximum

Note: These are two **independent** `if` statements (not `elif`), so both are always evaluated.

Exercise 4: BMI Classification

This exercise applies `if-elif-else` with numeric range checking – similar to the grade classification example in Section 4.5.

Suggested Code

```
bmi = float(input("ใส่ค่า BMI: "))

if bmi < 18.5:
    print("ผอม (Underweight)")
elif bmi < 25:
    print("ปกติ (Normal)")
elif bmi < 30:
    print("น้ำหนักเกิน (Overweight)")
else:
    print("อ้วน (Obese)")
```

BMI Categories

BMI Range	Category
Below 18.5	Underweight (ผอม)
18.5 - 24.9	Normal (ปกติ)
25.0 - 29.9	Overweight (น้ำหนักเกิน)
30.0 and above	Obese (อ้วน)

 Because conditions are checked top-to-bottom, writing `bmi < 25` in the second branch implicitly means $18.5 \leq \text{bmi} < 25$, since the first branch already handled values below 18.5.

Operator Quick Reference



Comparison Operators

`==` equal, `!=` not equal, `>` greater, `<` less, `>=` greater-or-equal, `<=` less-or-equal. All return `True` or `False`.



Indentation Rule

Use exactly 4 spaces per level. All statements in the same block must be at the same indentation level. Never mix tabs and spaces.



Logical Operators

`and` – both must be true. `or` – at least one must be true. `not` – inverts the Boolean value. Used to combine multiple conditions.



Nesting Rule

Each nested level adds another 4 spaces. Keep nesting shallow (max 2 - 3 levels) for readability. Deep nesting is a sign to refactor.

Logical Operator Truth Tables

Understanding how `and`, `or`, and `not` behave for all combinations of inputs is essential for writing correct conditions.

`and` Truth Table

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

`or` Truth Table

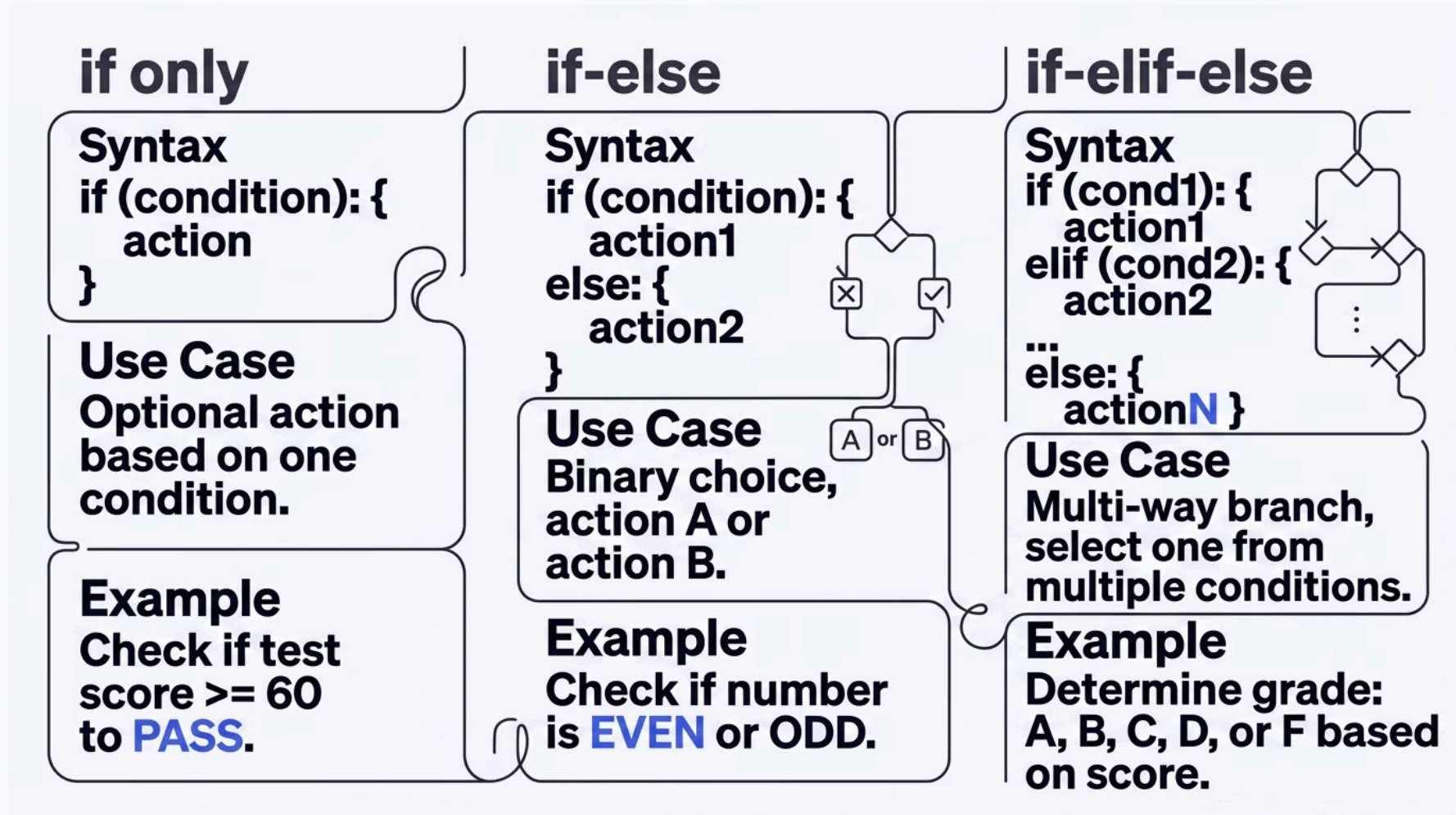
A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

`not` Truth Table

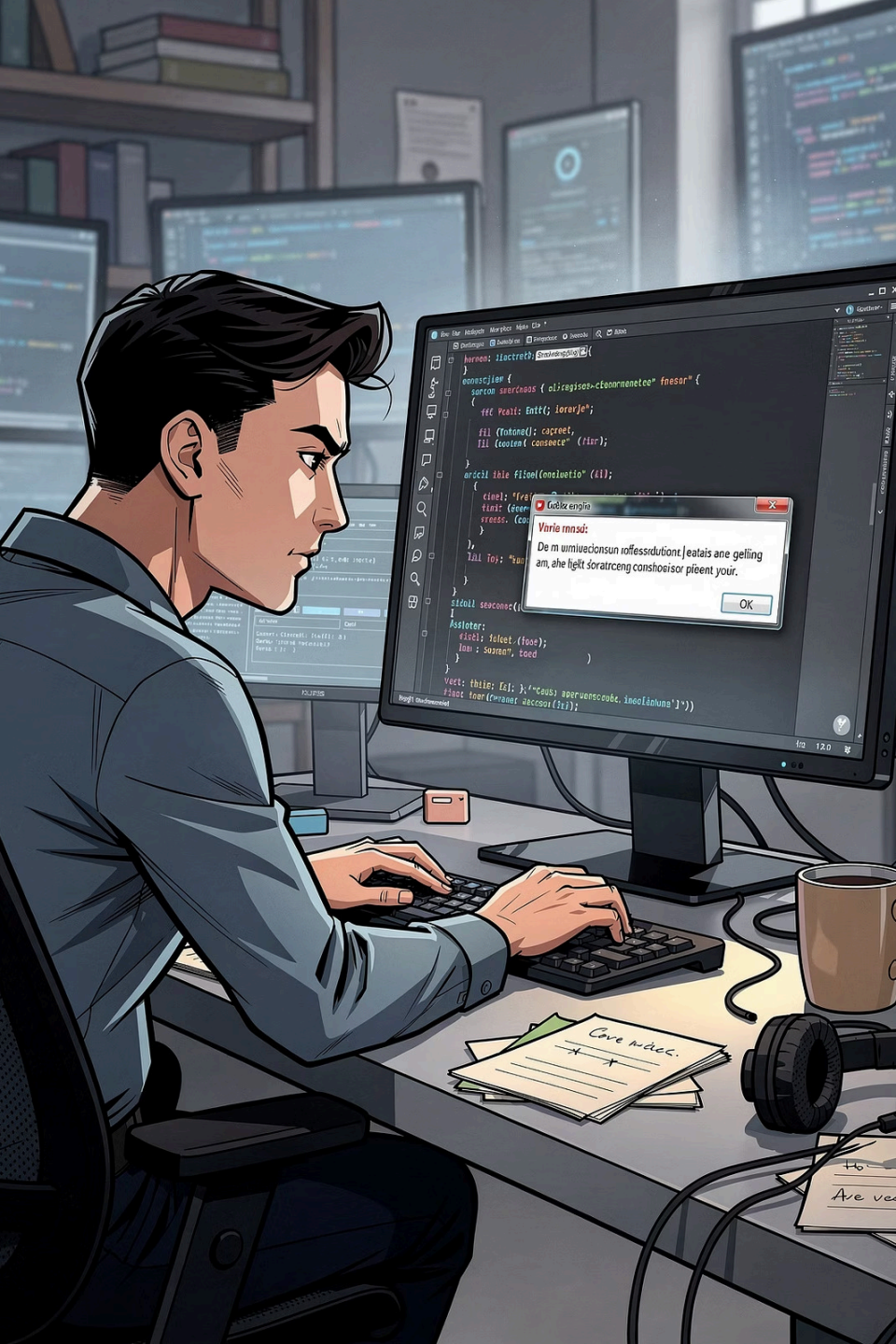
A	not A
True	False
False	True

 Short-circuit evaluation: Python stops evaluating `and` as soon as it finds a `False`, and stops evaluating `or` as soon as it finds a `True`.

Conditional Structures at a Glance



Select the structure that matches the number of distinct outcomes in your problem. Overusing `if-else` where `elif` is appropriate leads to redundant condition checks and harder-to-read code.



Common Mistakes to Avoid

Using `=` instead of `==`

Writing `if x = 5:` causes a `SyntaxError`. Always use `==` for comparison inside conditions. The single `=` is only for assignment.

Forgetting the Colon `:`

Every `if`, `elif`, and `else` line must end with a colon. Omitting it causes a `SyntaxError` immediately.

Inconsistent Indentation

Mixing tabs and spaces, or using inconsistent numbers of spaces, causes `IndentationError`. Use a code editor that converts tabs to 4 spaces automatically.

Overlapping `elif` Conditions

If conditions overlap, only the first matching branch executes. Ensure conditions are mutually exclusive or ordered correctly (e.g., check `>= 80` before `>= 70`).

Indentation: Correct vs Incorrect

✓ Correct Indentation

```
score = 75

if score >= 50:
    print("ผ่านเกณฑ์")
    print("ยินดีด้วย")

print("จบโปรแกรม")
```

Both `print` statements inside the block are indented by exactly 4 spaces. The final `print` is at column 0 – outside the block.

✗ Incorrect Indentation

```
score = 75

if score >= 50:
    print("ผ่านเกณฑ์") # IndentationError!
    print("ยินดีด้วย") # Unexpected indent
```

The first `print` has no indentation – Python expects at least one indented statement after the colon. The second has inconsistent indentation. Both cause errors.

✗ Python will refuse to run a program with indentation errors. Fix them before attempting to execute.

Chained Comparisons in Python

Python supports a unique and convenient syntax for range checks: **chained comparisons**. Instead of writing two separate conditions joined by `and`, you can chain them naturally.

Standard Form (using `and`)

```
if score >= 70 and score < 80:  
    grade = "B"
```

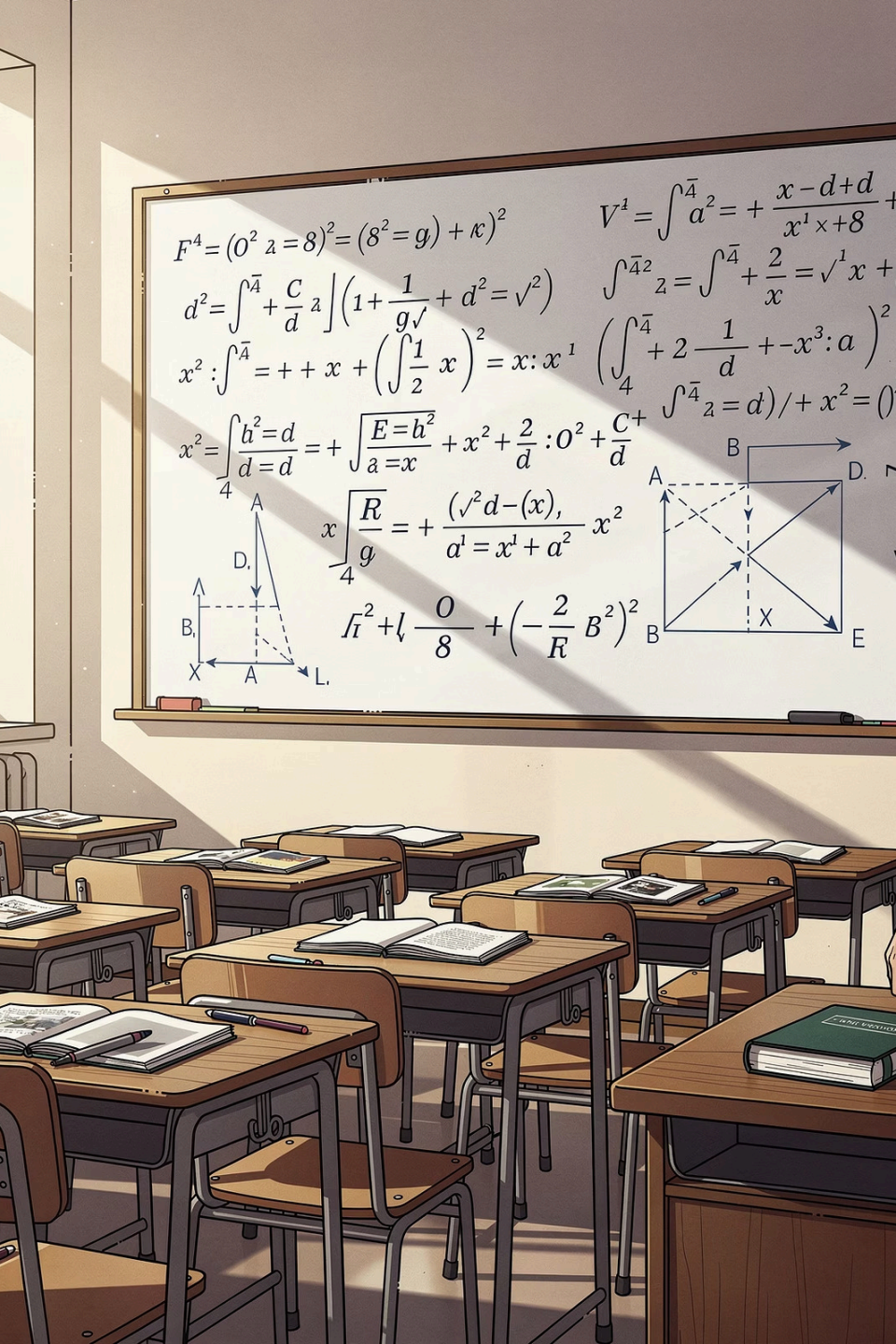
This is the explicit form – two separate comparisons combined with `and`. Works in all programming languages.

Chained Form (Python-specific)

```
if 70 <= score < 80:  
    grade = "B"
```

Python allows this mathematical notation directly. It is equivalent to the `and` form but more readable and concise. Both forms produce identical results.

① Chained comparisons are evaluated left to right. `a < b < c` means `a < b and b < c`. The middle value (`b`) is only evaluated once, which is both efficient and readable.



Applying Conditionals to Mathematics

Conditional structures are especially powerful in mathematical programming. Here are key mathematical applications covered in this chapter:

Number Classification

Determine sign (positive/negative/zero) and parity (even/odd) using `>`, `<`, and the modulo operator `%`.

Grade Boundaries

Map a continuous score to a discrete grade using ordered `elif` chains. The order of conditions matters critically.

Geometric Validation

Check the triangle inequality theorem using `and` to combine three conditions, then classify by side equality.

Divisibility Rules

Use `%` (modulo) with `==` to test divisibility. Combine with `and`/or `or` for complex rules like leap year detection.

The Modulo Operator in Conditions

The modulo operator `%` returns the **remainder** of integer division and is extremely useful in conditional expressions for mathematical programs.

Even/Odd Test

`n % 2 == 0` → even number

`n % 2 != 0` → odd number

Divisibility Test

`n % k == 0` → `n` is divisible by `k`

Example: `y % 4 == 0` checks if year is divisible by 4

Leap Year Logic

`(y%4==0 and y%100!=0) or (y%400==0)`

Combines modulo with `and`, `or`, `not` for a multi-rule condition

Examples

`print(10 % 3)` # Output: 1 ($10 = 3 \times 3 + 1$)

`print(15 % 5)` # Output: 0 ($15 = 5 \times 3 + 0$, so divisible)

`print(7 % 2)` # Output: 1 (7 is odd)

f-Strings in Conditional Output

All examples in this chapter use **f-strings** (formatted string literals) to produce readable output. An f-string starts with `f"..."` and embeds variable values using `{variable}` inside the string.

Syntax

```
name = "Alice"  
score = 85  
print(f"Hello, {name}! Your score is {score}.")
```

Output

```
Hello, Alice! Your score is 85.
```

Used Throughout This Chapter

- `print(f"{num} เป็นจำนวนคู่")` – even/odd program
- `print(f"เกรดที่ได้คือ {grade}")` – grade program
- `print(f"{y} เป็นปีอธิกสุรทิน")` – leap year program
- `print(f"ค่ามากที่สุดคือ {big}")` – maximum program

F-strings make output messages clear and informative by embedding computed values directly into the text without string concatenation.

Chapter 4 – Complete File Index

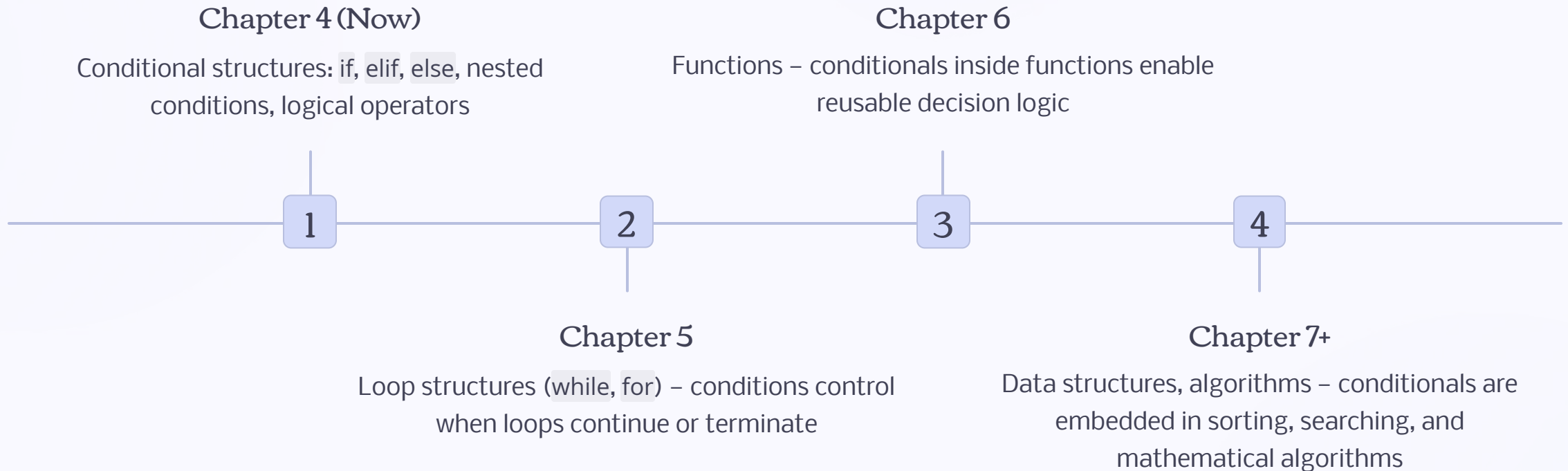
All Python files introduced in this chapter, with their purpose and key concepts demonstrated:

File	Topic	Key Concepts
04logical.py	Logical Operators	and, or, not
04if.py	Basic if Statement	if, indentation, block scope
04ifelse.py	if-else (Even/Odd)	if-else, modulo %
04grade.py	Grade Classification	if-elif-else, ordered conditions
04nested.py	Nested if (Sign + Parity)	Nested if, multi-level decisions
04triangle.py	Triangle Classification	Nested if, and, equality chains

- ✔ All files follow the naming convention 04xxx.py indicating Chapter 4. Practice by typing each program from scratch rather than copying – this builds muscle memory for syntax and indentation.

Connections to Future Topics

The conditional structures learned in this chapter are foundational to all subsequent programming topics in this course.



i Mastering conditional structures now will make every future chapter easier. Loops, functions, and algorithms all rely on the same Boolean logic and comparison operators introduced here.

Chapter 4 Complete

You have now studied all the core conditional structures in Python. Here is what you have mastered:

Expressions

Comparison and logical operators to build `True/False` conditions

Branching

`if`, `if-else`, and `if-elif-else` for 1, 2, and N-way decisions

Indentation

Python's mandatory block-scoping mechanism – 4 spaces per level

Nesting

Multi-level decisions for complex, hierarchical problems

- ✔ Complete all four end-of-chapter exercises before moving to Chapter 5. Writing code by hand is the most effective way to internalize these concepts.

